Practical - 10

Practical Title: Secure Login/Logout with Sessions, Role-Based Access, Timeout, and Remember-Me Token

Problem Definition: Develop secure login and logout functionality using PHP sessions. Verify password using password_verify(), regenerate session ID after login, restrict dashboard access, implement role-based redirection, session timeout, and secure remember-me token.

A. Theory & Analysis

1. Are sessions started and destroyed securely?

Yes. Sessions are initialized with session.cookie_httponly enabled, which prevents client-side JavaScript from accessing session cookies, mitigating cookie-theft via XSS. Upon successful login, session_regenerate_id(true) issues a new session ID to defend against session fixation. On logout, $_SESSION is cleared and session_destroy() terminates the session.

2. Is password_verify() used correctly?

Yes. In php/login_process.php, the script retrieves the stored hash for the user and verifies the submitted password using password_verify($password, $user['password_hash']), which performs timing-safe hash comparison.

3. Is unauthorized dashboard access blocked?

Yes. The check_auth() function in php/auth.php checks $_SESSION['user_id']. If no valid session exists, the script redirects the user to login.html?msg=unauthorized. Students attempting to open admin pages are redirected with an access denied alert.

4. Is session timeout implemented?

Yes. An inactivity timeout of 30 minutes (1800 seconds) is enforced by comparing time() against $_SESSION['last_activity']. If the time difference exceeds the threshold, the session is cleared and the user is redirected to login.html?msg=timeout.

5. Are admin and student users redirected to appropriate dashboards?

Yes. Successful authentication routes administrators to pages/admin_dashboard.php and students to pages/dashboard.php. If the user checked 'Remember me', a random token is hashed with SHA-256 and saved in the database for automatic login.

B. Implementation Details

I developed session authentication in php/auth.php, php/login_process.php, and php/logout.php. The login interface handles credential verification, session ID regeneration, 30-minute inactivity timeout, role redirection, and remember-me tokens stored in users.remember_token_hash. I verified login, failed credentials, logout, session expiration, and unauthorized access blocking.

C. Evidence of Work

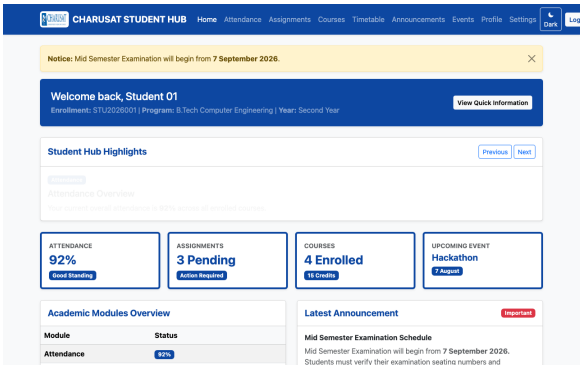


Fig 1: Authenticated Student Dashboard with Verified Active Session

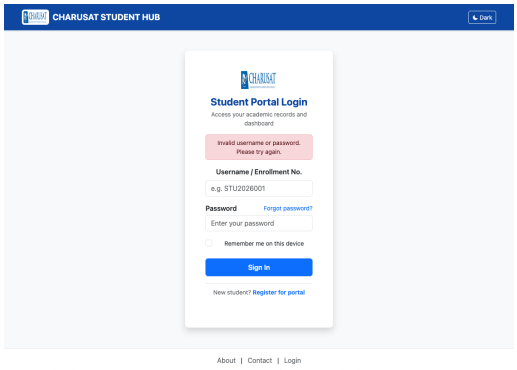


Fig 2: Failed Login Rejection Banner for Invalid Username or Password

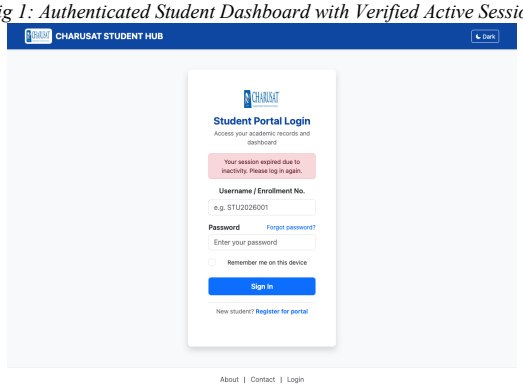


Fig 3: Session Expiration Alert After Inactivity Threshold Redirection

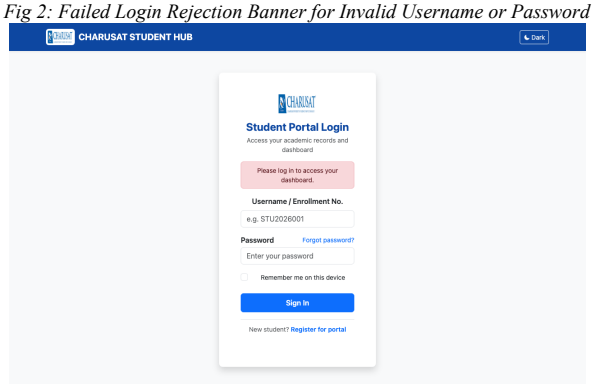


Fig 4: Unauthorized URL Access Blocked with Informative Alert Banner

D. Repository Link

GitHub Repository: https://github.com/neel92654/WDF_A_ARP

E. Learning Outcomes

I learned to build secure session authentication in PHP, including password verification, session fixation prevention, role-based access control (RBAC), automatic timeouts, and hashed remember-me tokens.